



# A Wait-Free Universal Construction for Large Objects

Andreia Correia

University of Neuchâtel, Switzerland  
andreia.veiga@unine.ch

Pedro Ramalhete

Cisco Systems, Switzerland  
pramalhe@gmail.com

Pascal Felber

University of Neuchâtel, Switzerland  
pascal.felber@unine.ch

## Abstract

Concurrency has been a subject of study for more than 50 years. Still, many developers struggle to adapt their sequential code to be accessed concurrently. This need has pushed for generic solutions and specific concurrent data structures.

Wait-free universal constructions are attractive as they can turn a sequential implementation of any object into an equivalent, yet concurrent and wait-free, implementation. Existing universal constructions either maintain a per-thread replica of the object, or make a new copy of the object on every operation, with both approaches yielding low performance.

To overcome these limitations, we have designed CX, a multi-instance-based wait-free universal construction that shares replicas among threads using reader-writer locks.

CX performs significantly better than existing wait-free constructions for our tested sequential data structures. Moreover, on read-oriented workloads, CX rivals hand-written lock-free and wait-free data structures. Our universal construction is the first to provide integrated wait-free memory reclamation.

**CCS Concepts** • Theory of computation → Distributed computing models; Concurrent algorithms.

**Keywords** universal construction, wait-free, memory reclamation

## 1 Introduction

A wait-free universal construction (UC) is a generic mechanism meant to provide concurrent wait-free access to a sequential implementation of an object or group of objects, e.g., a data structure. In other words, it takes a *sequential implementation* of an object and provides a concurrent implementation with wait-free progress [10].

In 1991, Herlihy [10] proposed the first wait-free UC for any number of threads. Herlihy's universal construction

keeps a queue of all operations ever applied. For every new operation, the operation's description is added to the queue and afterwards, all operations in the queue are applied to the initial state of the object. A relatively simple improvement of Herlihy's original approach, is to have each thread keep a local replica of the state of the object together with a reference to the last applied operation. Henceforth, to obtain the current state of the object, a thread only needs to update its replica from that point on. Even so, this approach would be unsuitable for practical usage as the overall work would amount to executing the original operation  $t$  times, where  $t$  is the number of threads. Furthermore, this algorithm does not provide a wait-free solution to memory reclamation and therefore, memory may be exhausted, creating a blocked execution due to allocation failure.

In 1992, Herlihy proposed a generic recipe [11] to be applied on a sequential data structure, which relies on maintaining an up-to-date instance that all threads copy from. In addition, Herlihy's combining consensus [11] establishes which group of operations are to be applied on the replica. Later, this generic recipe was turned into a UC, P-Sim [9]. In this approach, at least one copy of the object is made per  $t$  concurrent operations.

Our UC, named CX, utilizes a wait-free queue, similarly to Herlihy's wait-free UC. The first new idea in CX is that, instead of each thread having its own replica of the object, we keep a set of replicas that all threads may gain access to. In particular, we show that in the worst case,  $2t$  replicas always suffice to guarantee wait-freedom. Each replica has an associated reader-writer lock. For the sake of providing wait-free progress, the trylock methods of the reader-writer locks must execute in a finite number of steps and one of the competing threads must succeed in acquiring it. The usage of reader-writer locks enables CX to keep, at all times, the most recent replica protected with a shared lock, allowing multiple read-only operations to execute simultaneously. This design avoids serialization between read-only and mutative operations, making CX the first UC with parallel execution for read-only operations.

The actual execution of read-only operations is done in isolation without any synchronization, apart from the initial lock acquisition. This is an advantage when compared to hand-made lock-free data structures that share one single instance among all threads, and it is also the main reason why CX outperforms hand-made data structures in read-mostly workloads. After acquiring the exclusive lock on a replica, mutative operations also execute in isolation, although they

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

PPoPP '20, February 22–26, 2020, San Diego, CA, USA

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-6818-6/20/02...\$15.00

<https://doi.org/10.1145/3332466.3374523>

do so at the expense of serializing with all other mutative operations, seen as they have to add their operation in the queue. However, because all threads publish an operation before its actual execution, it is possible for a bulk of operations to be executed by a single thread. Meanwhile, a competing thread can simply return the result of the operation without even attempting its execution. Compared with previous UCs, the intuition behind the performance improvement for update operations is that a smaller number of replicas can be used and kept up-to-date by different threads. Even so, as the number of threads increase, there is no improvement in throughput for mutative operations simply because they are all serialized. This translates into lower performance for intensive to medium write workloads, when compared with hand-made data structures supporting disjoint access operations. In addition CX has a much higher memory footprint than hand-made data structures because it may have up to  $2t$  replicas of the object. On the other hand, P-Sim can have up to  $t^2$  objects in memory. For systems where memory is scarce and resilience to thread failure is required, hand-made lock-free data structures based on a single replica are a better solution.

CX is the first UC with integrated memory management. In order to prevent unbounded memory usage, older nodes in CX's operation queue must be reclaimed. Reclaiming these older nodes may prevent a stale replica from being brought up-to-date, given that the descriptions of the older operations required to update the stale replica, are no longer available. When this happens, a complete copy of the most recent replica is made, using the copy constructor of the sequential object. The only objects in CX that need a memory reclamation technique, are the nodes of the queue. For that, we use a combination of Reference Counters [25] and Hazard Pointers [19] with wait-free progress.

In this paper, we focus on improving UC state of the art, with the goal of addressing their main limitations in terms of performance and usability, which made them so far impractical for real-world applications. We introduce CX, a UC with linearizable operations and wait-free progress that does not require any annotation of the underlying sequential implementation.

CX provides fast and scalable read-only operations by exploiting their *disjoint access parallel* [14] nature.

CX is the first practical wait-free UC, written in portable C++, with integrated wait-free memory reclamation and high scalability for read-mostly workloads. Our experimental evaluation shows that CX outperforms other state-of-the-art UCs and performs competitively with hand-written non-blocking data structures in some workloads.

The rest of the paper is organized as follows. We first discuss related work in §2. We then present the CX algorithm in §3, before proving its correctness in §4. We perform an in-depth evaluation of CX and its memory usage in §5. Finally we conclude in §6.

## 2 Related Work

Software Transactional Memory [12, 24] and Universal Constructions are two generic concurrency techniques. Unlike UCs, software transactional memory (STM), has transactional semantics, allowing the user to make an operation or group of operations seem *atomic* and providing *serializability* between transactions [12]. STMs and UCs present two separate approaches to developers when it comes to dealing with concurrent code. Both approaches allow the end user to reason about the code as if it were sequential. STMs *instrument* the loads and stores on the sequential implementation. STMs may also require type annotation, function annotation or replacement of allocation/deallocation calls with equivalent methods provided by the STM.

We observe that the UC literature can be classified into two groups of algorithms, UCs that do not require instrumentation, and UCs that do. *Non-instrumenting* UCs require no annotation of the sequential implementation, allowing the developer to *wrap* the underlying object and creating an equivalent object with concurrent access for all of its methods. *Instrumenting* UCs require the developer to annotate and modify the sequential implementation, similar to what must be done for an STM. This annotation implies effort from the developer and is prone to errors. In addition, the fact that annotation is required at all, makes it difficult or infeasible to use legacy code or data structures provided by pre-compiled libraries (e.g., `std::set` and `std::map`) because it would require modifying the library's source code. In this paper, we focus on non-instrumenting UCs with the goal of addressing their main limitations in terms of performance and usability, which made them so far impractical for real-world applications.

In 1977, Lamport [17] was the first to attack the problem of non-blocking access to shared data and later in 1983, Peterson [21] provided several solutions to what he called the *concurrent reading while writing* problem. One of these solutions uses two instances of the same data and guarantees wait-free progress for both reads and writes, allowing multiple readers and a single writer to access simultaneously any of the two instances. However, this approach is based on *optimistic concurrency*, causing read-write races, which has troublesome implications in terms of atomicity, memory reclamation and invariance conservation.

In 1991, Herlihy [10] proposed the first wait-free UC for any number of threads. This UC requires no annotation or modification to the sequential implementations and is therefore a non-instrumenting UC. On the next year he provided a wait-free generic recipe and a combining consensus [11].

Since then, several wait-free UCs have been proposed [2, 3, 6, 8, 9]. Researchers have attempted to address the problem of applying wait-free UCs to large objects [1, 3, 10] though none has succeeded in providing a generic solution [23].

Anderson and Moir [3] have proposed a technique designed to work well with large objects. This algorithm is an instrumenting UC because it requires the end user to write a sequential procedure that treats the object as if it was stored in a contiguous array, implying adaptation or annotation of the sequential implementation. This technique is also not universally applicable to all data structures.

Chuong et al. [6] have shown a technique that makes a copy of each shared variable in the sequential implementation and executes the operations on the copy. Although this technique can operate at the level of memory words, it is vulnerable to race conditions from different (consecutive) operations that modify the same variables. Even if a CAS is used to modify these variables, ABA issues can still occur. This word-based approach requires instrumentation of the user code.

Fatourou and Kallimanis [9] have designed and implemented P-Sim, a highly efficient wait-free and non-instrumenting UC based on fetch-and-add and LL/SC. P-Sim relies on an up-to-date instance that all threads copy from, using Herlihy's combining consensus and a generic approach [11] to establish which operations are to be applied on the replica, independently of whether these are read or write operations. In the best-case scenario, one copy of the entire object state is made per  $t$  concurrent operations. In the worst-case, two copies are done per operation. Unfortunately, P-Sim is impractical for large objects.

More recently, Ellen et al. [8] have shown a wait-free UC based on LL/SC. Their technique provides disjoint access parallel operations with the requirement that all data items in the sequential code are only accessed via three special instructions `CreateDI`, `ReadDI` and `WriteDI`, to create a *data instance* or node in the data structure, read the node or write to a node, respectively. Their approach implies the need for instrumentation of the sequential implementations similar to an STM. No implementation has been made publicly available.

### 3 CX Algorithm

The CX wait-free construction uses a wait-free queue where mutations to the object instance are placed, much like Herlihy's wait-free construction, though instead of each thread having its own replica of the object, there are a limited number of replicas that all threads can access. The access to each of these copies is protected by a reader-writer lock, which can be acquired by multiple reader threads in *shared mode*, whereas only one writer thread can get the lock in *exclusive mode*. The reader-writer lock used in CX must guarantee that, when multiple threads compete for the lock using the trylock methods, at least one will succeed and obtain the lock. This property, named *strong trylock* [7], combines *deadlock freedom* with linearizable consistency and wait-free progress.

The CX construction (see Figure 1) requires  $2t$  Combined instances, where  $t$  is the number of threads and each Combined instance aggregates several attributes associated with a replica. As such, CX is composed of: (i) `curComb`: a pointer to the most up-to-date Combined instance; (ii) `tail`: a pointer to the last node of the queue; and (iii) `combs`: an array of Combined instances.

In turn, a Combined instance consists of: (i) `head`: a pointer to a Node on the queue of mutations; (ii) `obj`: a replica of the data structure or object that is up to date until `head`; and (iii) `rwlock`: an instance of a reader-writer lock that protects the content of the Combined instance.

Finally, a Node on the queue of mutations holds: (i) `mutation`: a function to be applied on the object; (ii) `result`: the value returned by the update function, if any; (iii) `next`: a pointer to the next Node in the mutation queue; (iv) `ticket`: a sequence number, as well as some other fields for internal use. The definitions for the main data structures of CX are shown in Algorithm 1.

---

#### Algorithm 1 CX data structures

---

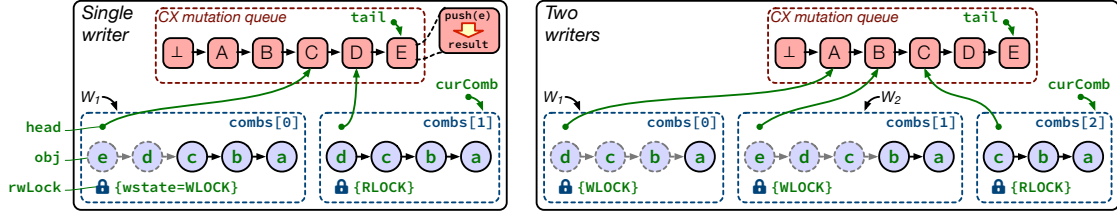
```

1 template<typename C, typename R = uint64_t>
2 class CX {
3     const int t; // Maximum number of threads
4     std::atomic<Combined*> curComb {nullptr};
5     // The first Node on the queue contains a dummy operation
6     Node* sentinel = new Node( [] (C* c) {return R{}; } );
7     std::atomic<Node*> tail {sentinel};
8     Combined combs[2*t];
9     struct Node {
10         std::function<R(C*)> mutation; // Not a pointer
11         std::atomic<R> result; // Not a pointer
12         std::atomic<Node*> next {nullptr};
13         std::atomic<uint64_t> ticket {0};
14         std::atomic<int> refcnt {0}; // Memory reclamation
15         const int enqTid; // Used internally by queue
16     };
17     struct Combined {
18         Node* head {nullptr};
19         C* obj {nullptr};
20         StrongTryRWRL rwLock {};
21         void updateHead(Node* mn) {
22             mn->refcnt.fetch_add(1);
23             if (head != nullptr) head->refcnt.fetch_add(-1);
24             head = mn;
25         }
26     };
27 };
```

---

Figure 1 illustrates the data structures and principle of CX on a concurrent stack. Mutative operations in the wait-free queue are represented by rounded rectangles, with node A corresponding to operation `push(a)`. The stack stores its elements in a linked list of nodes (circles), with dashed lines indicating the nodes that are not yet added to the specific instance of the data structure.

The CX construction relies on the object replicas present in the `combs[]` array. Initially, one of the Combined instances in the array, holds an initialized object `obj` and its head pointer



**Figure 1.** Illustration of CX's principle on two scenarios with one (left) and two (right) writers pushing elements a through e in a shared stack.

refers to the sentinel node  $\perp$  of CX's wait-free queue of mutations. The other instances have both obj and head set to null.

To improve *readers* performance, CX has distinct code paths for *readers* and *updaters*. Readers call `applyRead()`, which tries to acquire the shared lock on the reader-writer lock of the current Combined instance, `curComb`. `curComb` always references the most up-to-date replica which is, at all times, protected by a shared lock. Updaters call `applyUpdate()`, which scans the `combs[]` array and attempts to acquire the exclusive lock on the reader-writer lock of one of the Combined instances (which is guaranteed to succeed after a maximum of  $\text{numReaders} + 2 \times \text{numUpdaters}$  trials).

An updater thread has to account for a possible read operation that will be executed in the most up-to-date replica, which is referenced by `curComb`. The updater is responsible for leaving `curComb` referencing a replica that contains its mutative operation, and this replica is left with a shared lock held so as to protect it from being acquired in exclusive mode by updater threads, including itself. When a copy of the object is required, the copy procedure will try to acquire `curComb` in shared mode and copy its obj while holding the shared lock, therefore guaranteeing a consistent replica. This implies that two Combined instances may be required for each updater thread: the original replica and the new replica. If we consider that the construction will be accessed by at most  $\text{numReaders}$  dedicated readers and  $\text{numUpdaters}$  dedicated updaters, then the maximum number of Combined instances in use at any given time, will be one per reader plus two per updater, *i.e.*,  $\text{numReaders} + 2 \times \text{numUpdaters}$  (or  $2t$  if every thread can potentially read and update the data structure).

Once an updater thread secures a Combined instance with exclusive access and ensures it has a replica of the object, the updater is responsible for applying all the mutations present in the mutation queue, following the sequential order from the head of the Combined instance until its own mutative operation, which was previously added to the queue. Each node has a ticket that simplifies the validation in case the mutation has already been applied and head is more recent than the node  $N$  containing the mutative operation to be made *visible*. The concept of visibility refers to a state of the

object, where effects of operations on the object are available to all threads. After the Combined instance is brought up-to-date with a replica of the object containing the updater's mutation, the updater thread has to make its mutation visible to other threads by ensuring that `curComb` advances to a Combined instance whose head has a ticket greater than or equal to  $N$ 's ticket.

We define a valid replica of the object, as being an instance that can be brought up to date, applying all the mutations starting from the head of the Combined where the replica is stored, until  $N$ 's mutation. An invalidation of a replica may occur when there is memory reclamation of the queue's nodes, *i.e.*, a node is "self-linked". In case a replica is invalidated, a new copy can be made from `curComb`.

Consider Figure 1 to better understand how mutations are propagated to the available replicas. In the left figure, a single writer  $W_1$  has been pushing values a, b, c and d, and is in the process of executing `push(e)`. The mutative operation has already been inserted at the tail of CX's wait-free queue but not yet applied to the stack. At that point `curComb` points to `combs[1]`, which holds an up-to-date stack (with all 4 elements inserted) protected by a shared lock. Hence the writer cannot use this instance and instead acquires `combs[0]` in exclusive mode. The next steps for the writer will be to apply the operations starting from the head, *i.e.*, push d and e, to bring the data structure up to date, update head to point to the last applied mutation (node E), downgrade the lock to read mode, and finally atomically set `curComb` to point to `combs[0]` with a compare-and-set (CAS). The right figure presents a similar scenario but with two concurrent writers,  $W_1$  and  $W_2$ , respectively executing `push(d)` on `combs[0]` and `push(e)` on `combs[1]`. The order of operations is determined by their position in the wait-free queue, *i.e.*, d is inserted before e and both writers will apply the operations on the Combined instances in this order.

### 3.1 Algorithm Walkthrough

The core of the CX algorithm resides in the `applyUpdate()` mutative operation, shown in Algorithm 2. The main steps of the algorithms are:

1. Create a new Node `myNode` with the desired mutation and insert it in the queue (line 18).

**Algorithm 2** CX ApplyRead and ApplyUpdate pseudo-code

```

1: function APPLYREAD(readFunc):
2:   for i ← 0, MAX_READ_TRIES + t do
3:     if i = MAX_READ_TRIES then
4:       myNode ← enqueue(readFunc)
5:       hp.protectPtr(HP0, myNode)
6:       combIdx ← curComb
7:       comb ← combs[combIdx]
8:       if comb.rwlock.sharedTryLock() then
9:         if combIdx = curComb then
10:          ret ← readFunc(comb.obj)           ▷ Function call
11:          comb.rwlock.sharedUnlock()
12:          return ret
13:          comb.rwlock.sharedUnlock()
14:       end for           ▷ A writer must have completed its operation. . .
15:       return myNode.result           ▷ . . .and the result is in myNode
16: end function

17: function APPLYUPDATE(updFunc):
18:   myNode ← enqueue(updFunc)               { 1 }
19:   hp.protectPtr(HP0, myNode)
20:   tkt ← myNode.ticket
21:   idx ← exclusiveTryLock()                 { 2 }
22:   c ← combs[idx]
23:   mn ← c.head
24:   if mn ≠ null ∧ mn.ticket ≥ tkt then
25:     c.rwLock.exclusiveUnlock()
26:     return myNode.result
27:   comb ← null
28:   combIdx ← -1
29:   while mn ≠ myNode do
30:     if mn = null ∨ mn = mn.next then           { 3 }
31:       combIdx ← getCombined(tkt)
32:       if comb ≠ null ∨ combIdx = -1 then
33:         c.head ← mn
34:         c.rwLock.exclusiveUnlock()
35:         return myNode.result
36:       comb ← combs[combIdx]
37:       mn ← comb.head
38:       c.updateHead(mn)
39:       delete c.obj
40:       c.obj ← new C(comb.obj)           ▷ Copy constructor
41:       comb.rwLock.sharedUnlock()
42:     else
43:       lnext ← hp.protectPtr(HP1, mn.next)
44:       if mn = mn.next then
45:         continue
46:       lnext.result ← lnext.updFunc(comb.obj) { 4 }
47:       hp.protectPtr(HP2, lnext)
48:       mn ← lnext
49:       c.updateHead(mn)                   { 4 }
50:       c.rwLock.downgradeToHandover()    { 5 }
51:       for i ← 0, t do
52:         combIdx ← curComb
53:         comb ← combs[combIdx]
54:         if ¬comb.rwlock.sharedTryLock() then
55:           continue
56:         if comb.head.ticket ≥ tkt then
57:           comb.rwlock.sharedUnlock()
58:           if comb ≠ curComb then
59:             continue
60:           break
61:         if curComb.cas(combIdx, idx) then           { 6 }
62:           comb.rwLock.handoverUnlock()
63:           node ← comb.head
64:           while node ≠ myNode do           ▷ Reclaim queue nodes
65:             lnext ← node.next
66:             tl_preRetired.add(node)         ▷ Place node in . .
67:             node ← lnext                   ▷ . . .thread-local circular buffer
68:           return myNode.result
69:           comb.rwLock.sharedUnlock()
70:         end for
71:         c.rwLock.handoverUnlock()
72:         return myNode.result
73:       end function
74:       ▷ Acquire shared lock on curComb and get Combined instance
75:       function GETCOMBINED(myTicket):
76:         for i ← 0, t do
77:           combIdx ← curComb
78:           comb ← combs[combIdx]
79:           if ¬comb.rwlock.sharedTryLock() then
80:             continue
81:           lhead ← comb.head
82:           tkt ← lhead.ticket
83:           if tkt < myTicket ∧ lhead ≠ lhead.next then
84:             return combIdx
85:           if tkt ≥ myTicket then
86:             return -1
87:         end for
88:         return -1
89:       end function

```

2. Acquire an exclusive lock on one of the Combined instances in the `combs[]` array. `exclusiveTryLock()` performs a sequential search from 0 to  $2t$  (line 21).
3. Verify if there is a valid replica of the data structure in `c` and make a copy if necessary (lines 30 to 41).
4. Apply all mutations starting at head of the Combined instance until `myNode` (lines 29 to 48), and update head to point to `myNode` (line 49).
5. Downgrade lock on `c` (line 50).
6. CAS `curComb` from its current value to the just updated Combined instance (line 61). Upon failure, retry CAS until successful or until head of the current `curComb` instance is *after* `myNode`.

When applying a mutation to the underlying object, the first step is to create a new node with the mutation (line 18 of

Algorithm 2) and insert it in CX's queue. Each node contains a mutation field that stores the mutation. A monotonically increasing ticket is assigned to the node to uniquely identify the mutation (line 18).

The next step consists in finding an available Combined instance on which to apply the new mutation. To that end, the thread must acquire a Combined's lock in exclusive mode (line 21). The StrongTryRWRI [7] reader-writer lock provides a *strong* `exclusiveTryLock()` method, guaranteeing that the lock will be acquired in at most  $2t$  attempts.

If the locked Combined instance has an invalidated or null `obj` (line 30), we need to make a copy of the current object. To do so, we first acquire the shared lock on `curComb` (line 31), before updating head (line 38) and copying `obj` (line 40). It is worth mentioning that copy-on-write (COW)

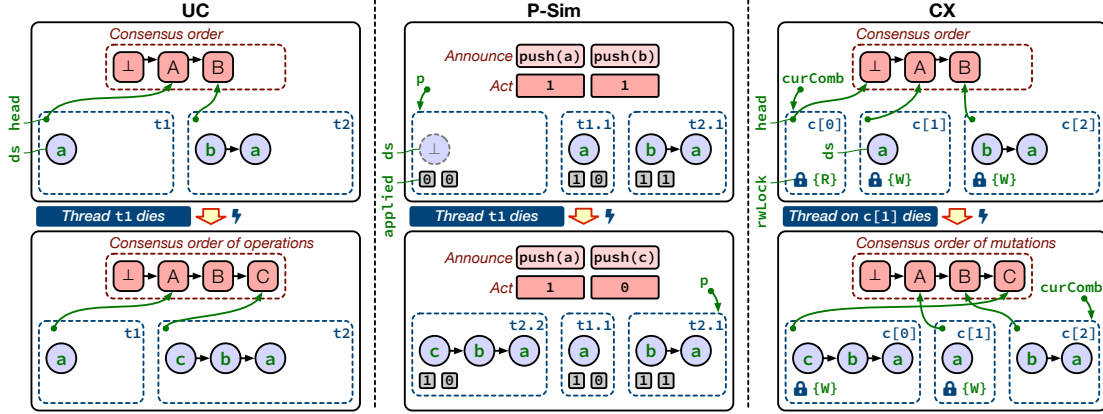


Figure 2. Comparison of UC [10], P-Sim [9] and CX.

based techniques usually make one such copy for every mutative operation, while CX does this *once* for every new used Combined (of which there are  $2t$ ) plus the number of times a replica is invalidated.

Next, we apply the mutations on `obj`, starting from the corresponding head node (line 23) until our newly added node is found (line 29), always saving the result of each mutation in the corresponding node `result` (line 46). The rationale for saving the result is that, if another thread calling `applyUpdate()` sees that its own mutation is already visible at `curComb`, it can directly return the result of the mutation (lines 26, 35, 69 and 73) without having to actually execute the previous mutations in the queue. This approach implies that multiple threads may be write-racing the same value into node `result`, which means that it must be accessed atomically, further implying that `R` must fit in a `std::atomic` data type to ensure wait freedom.

After the mutations have been applied, we downgrade the lock on `c` and advance `curComb` with a CAS (line 61) so as to make the current and previous mutations visible to other threads: `curComb` will now reference a Combined instance that contains the effects until `head.ticket`, also `curComb` will be protected by a shared lock to guarantee the instance is always available to execute a read operation. In addition, `curComb` always transitions to a Combined instance with a `head.ticket` higher than the previous one, thanks to the test at line 56. This guarantees that operations whose effects are visible on `curComb` will remain visible. Finally, we can unlock the unneeded Combined instance to make it available for exclusive locking by other threads searching for their `c`, before returning the result of our mutation.

As mentioned previously, a read operation will attempt to acquire a shared lock in the most up to date instance `curComb`. The reader may be unsuccessful in acquiring the lock if an updater has already acquired the lock. Such a situation can occur if, between the load at line 6 and the call

to `sharedTryLock()` at line 8, the `curComb` advances to another Combined instance and an updater takes the exclusive lock on the previous instance. This results in a lock-free progress condition for read operations, as it would only fail to progress in case an updater had made progress. To guarantee wait-free progress, the reader must publish its operation in the queue after `MAX_READ_TRIES` attempts (line 4), but it is not required to apply all mutations up to its own operation. After a maximum of  $t$  transitions of `curComb`, the reader's operation will be processed by an updater thread and become visible. If there is no updater thread to process the read operation, then there is also no updater thread to block the reader thread from acquiring the lock on `curComb`.

To better understand the differences between Herlihy UC [10], P-Sim [9] and CX, we show in Figure 2 a comparison example where two writer threads are pushing elements to a stack. Both threads reach a consensus where the operation `push(a)` will be executed followed by `push(b)`. Thread  $T_1$  terminates abruptly while executing operation `push(a)`, while thread  $T_2$  continues execution pushing element `c` to the stack. With Herlihy's UC, every thread maintains its own replica of the data structure and each replica has to know the exact order in which the operations must be executed. Because  $T_2$  has no way to know if  $T_1$  has died or is just delayed, the order of the operation from `A` onwards has to be kept, which will cause it to grow indefinitely, eventually exhausting the system's memory.

P-Sim uses a copy-on-write approach with Herlihy's combining consensus [11]. Each thread performs a copy of the object referenced by `P` and applies to its replica all operations newly published in the `announce` array. There is no consensus order that all threads agree upon; instead, the thread that is able to transition `P` to its replica is the one establishing the order of the operations. In case thread  $T_1$  dies, its replica (`t1.1`) is left unreclaimed but this has no impact on the execution of thread  $T_2$ .

CX has a pool of  $2t$  replicas of the data structure that all threads can use to execute their operation, and uses a *turn queue* for consensus [22]. In the example, there are four available replicas in the pool but the two threads only used three Combined instances. In the scenario where a thread dies while executing on the second one ( $c[1]$ ), this instance is no longer available to the remaining threads. But in case the thread died after releasing the exclusive lock, then the instance would remain available to be used. Also, the order of the mutations can be disposed off up until  $\text{curComb.head}$ , because at any given time there is an object referenced by  $\text{curComb}$ , that is protected by a shared lock, from which any thread can execute a copy. In addition, there are at most  $t$  operations after  $\text{curComb.head}$  that remain to be executed.

### 3.2 Reader-Writer Lock with Strong Trylock

Access to each Combined instance and, consequently, to each replica of the object is managed by a reader-writer lock,  $\text{Combined.rwlock}$ . In order to ensure wait-free progress, the reader-writer lock has to guarantee that from all the threads competing for the lock, at least one will acquire it. This property is sometimes called *deadlock freedom for trylock*. Furthermore the trylock methods must complete in a finite number of steps [7]. Not only is this strong guarantee needed, but it is also desirable to have a reader-writer lock with good scalability for read-mostly workloads, so as to provide high throughput in the  $\text{applyRead}()$  method. Based on these requirements, we chose to use the  $\text{StrongTryRWRI}$  reader-writer lock proposed in [7]. This lock's high scalability is capable of matching other state of the art reader-writer locks [5] while providing  $\text{downgrade}()$  functionality and strong trylock properties.

---

#### Algorithm 3 Addition to $\text{StrongTryRWRI}$ [7]

---

```

1 void handoverLock() { wstate.store(RLOCK); }
2 void handoverUnlock() { wstate.store(NOLOCK); }
3 void downgradeToHandover()
4   { wstate.store(RLOCK); ri.abortRollbacks(); }
```

---

In addition, CX requires *lock handover* between different threads when in shared mode. In CX, the  $\text{rwlock}$  of each Combined instance can be in one of four logical states: unlocked; shared; exclusive; or handover. The handover state, which is not typical in  $\text{rwlock}$  implementations, represents a state in which the lock is left in shared (read-only) mode without any thread actually using it, with the purpose of preventing writers from acquiring the lock in exclusive mode,  $\text{handoverLock}()$  or  $\text{downgradeToHandover}()$  will leave the lock in handover state. The  $\text{rwlock}$  implementation allows the unlock of the shared mode by a different thread from the one which acquired the lock in shared mode,  $\text{handoverUnlock}()$ .

### 3.3 Memory Reclamation

Access to each replica is regulated by the reader-writer lock in the respective Combined instance. Consequently, invalidated replicas are deleted by the next updater thread that acquires the exclusive lock, which makes memory reclamation of replicas easy. Object reclamation in data structure replicas is trivial because modifications on those replicas are executed with exclusive access. However, this is not the case for the nodes of the queue.

The wait-free queue in CX is utilized to maintain consistency between concurrent threads but it has an impact in memory allocation and its subsequent memory reclamation. A few wait-free queues have been proposed in the literature (e.g., [9, 15, 22]). In our implementation, we use a variant of [22] notably because it provides a simple  $\text{enqueue}()$  method and supports wait-free memory reclamation.

Every Combined instance contains a head pointer, which references a node in the mutation queue. If this node is disposed of (or equivalently “self-linked”) during memory reclamation, this will invalidate the replica and force a future updater thread to make a new copy. This can have a negative impact in terms of performance on medium to large data structures. On the other hand, it can also be costly to apply all the mutations necessary to bring a very old replica up to date. There is an inherent trade-off in collecting the nodes of the queue and the consequent invalidation of existing replicas of the object.

The only objects in CX requiring a concurrent memory reclamation technique to track their lifetime, are the nodes in the queue of mutations. It may seem like the wait-free queue should be able to handle it all, unfortunately, there are peculiarities regarding memory reclamation of the queue's nodes. The most complex to solve, concerns the nodes referenced by the head of each Combined instance, because a node can only be retired after it becomes inaccessible.

When a new replica of the data structure is needed for a given Combined instance, it will be copied from the  $\text{curComb}$  instance together with its associated head pointer. This means that a scan of the heads of all the Combined instances in  $\text{combs}[]$  would result in an erroneous memory reclamation scheme, because while a thread is verifying the possible deletion of a node, a copy of its pointer may be made on the opposite direction of the scan, to a previously scanned head, thus leading to the deletion of a node that is still being referenced.

A solution to this specific problem is to add reference counting to hazard pointers [19]. Every node of the queue has a  $\text{refcnt}$  variable, which stores the number of references that the heads of all Combined instances have to that specific node. After a node is retired using hazard pointers, it will only be deallocated when its  $\text{refcnt}$  reaches zero. In case a replica is updated or a full copy is being made, the  $\text{refcnt}$  of the new head will be incremented and the  $\text{refcnt}$  of the

previous head of that Combined instance will be decremented. Because the copy procedure of a data structure requires that the updater thread first acquires a shared lock on the instance that it will copy from, this guarantees not only a consistent copy, but also that no other updater can be working on that instance. This in turn ensures that the `refcnt` for the node of the corresponding head is greater than zero and consequently that the node is not deleted.

When updating a replica, the queue of mutations must be traversed. We protect the traversal using hazard pointers. In the original hazard pointers algorithm, protecting a pointer consists of (i) reading the corresponding memory location, (ii) publishing the pointer read from there, and (iii) re-checking if the pointer is still valid, usually by re-reading the same memory location. In the general case, another thread may update the pointer between the first and third steps, causing re-execution and making this operation lock-free but not wait-free.

Let us now describe how this is not preventing wait-free progress for our specific queue. When traversing the CX queue, if the check after publishing finds a “self-linked” node, we restart the traversal loop, line 29. If the next of the node we are currently traversing has changed, then it must have been self-linked. This is the only possible pointer modification for a queue node. Finding a self-linked node during traversal implies that this replica is too old to be brought up to date and a full copy from the `curComb` replica must be made instead. The CX algorithm guarantees that a replica copied from `curComb` is at most  $t$  operations (nodes) away from the last enqueued operation. Once a fresh copy from `curComb` is made, if the subsequent traversal, of at most  $t$ , finds a self-linked node, then the operation must be visible at  $t$  and the result of the operation is now available in `myNode.result`. Ultimately, these invariants ensure that the operation has safe memory reclamation and wait-free progress.

The actual reclamation of the nodes is done by the thread that transitions `curComb`. A `curComb` transition always occurs between two replicas,  $Comb_i$  and  $Comb_j$ , where the first replica’s  $Comb_i.head$  references an older node in the queue than  $Comb_j.head$ . The thread responsible for this transition will initially place all the nodes of the queue from  $Comb_i.head$  to  $Comb_j.head$  in its thread-local circular buffer, and once the circular buffer is full, it will retire the oldest half of the circular buffer. For clarity, all memory reclamation instructions shown in Algorithm 1 and Algorithm 2 are displayed in a lighter blue color.

## 4 Correctness

We discuss the correctness of CX using standard definitions and notations for linearizability [13], which we briefly recall below. A concurrent execution is modeled by a history, *i.e.*, a sequence of events. Events can be operation invocations and responses, denoted respectively as  $op.inv$  and  $op.res$ . Each

event is labeled with the process and with the object  $O$  to which it pertains. A subhistory of a history  $H$  is a subsequence of the events in  $H$ . A response matches an invocation if they are performed by the same process on the same object. An operation in a history  $H$  consists of an invocation and the next matching response. An update operation may cause a change of state in the object, with a visible effect to other processes, while a read-only operation has no effects visible to other processes. An invocation is pending in  $H$  if no matching response follows it in  $H$ . An extension of  $H$  is a history obtained by appending responses to zero or more pending invocations in  $H$ , and  $complete(H)$  denotes the subhistory of  $H$  containing all matching invocations and responses. All references to specific lines of code in this section refer to Algorithm 2.

► **Definition 1.** (Happens before) *if  $op_1.res <_{hb} op_2.inv$  then  $op_1 <_{hb} op_2$ .*

► **Definition 2.** (Subhistory) *Given a history  $H$ , a subhistory  $S$  of  $H$  is such that if  $op_2$  belongs to  $S$  and  $op_1 <_{hb} op_2$  in  $H$ , then  $op_1$  belongs to  $S$  and  $op_1 <_{hb} op_2$  in  $S$ .*

► **Definition 3.** (Partial subhistory) *Given a history  $H$  and update operations  $op_1$  and  $op_2$ , a partial subhistory  $S$  of  $H$  is such that if  $op_2$  belongs to  $S$  and  $op_1 <_{hb} op_2$  in  $H$ , then  $op_1$  belongs to  $S$  and  $op_1 <_{hb} op_2$  in  $S$ .*

► **Definition 4.** (Linearizability) *A history  $H$  is linearizable if  $H$  has an extension  $H'$  and there is a legal sequential subhistory  $S$  such that: (i)  $complete(H')$  is equivalent to  $S$ ; and (ii) if an operation  $op_1 <_{hb} op_2$  in  $H$ , then the same holds in  $S$ .*

CX requires a linearizable wait-free enqueue method with linearizable traversal of the queue that provides a partial subhistory of the history  $H$ . The linearizable reader-writer lock must provide strong guarantees for the `sharedTrylock()` and `exclusiveTrylock()` methods. Both methods must satisfy the property of *deadlock-freedom*, *i.e.*, the critical section will not become inaccessible to all processes, and their invocation must complete in a finite number of steps.

We denote by  $Comb_i$  the  $i_{th}$  Combined instance of the `combs[]` array. At any given moment,  $Comb_i$  is in a state represented by the pair  $\langle O_{i,j}, head_{i,j} \rangle$ .  $O_{i,j}$  represents the  $i_{th}$  simulation of object  $O$  where  $j$  corresponds to the sequence number in a node of the wait-free queue. As such,  $O_{i,0}$  is the initialized object and  $O_{i,j}$  is the simulation of object  $O$  after sequentially applying all update operations up to the operation with sequence number  $j$ . We assume all  $Comb_i$  instances start in the initial state  $\langle O_{i,0}, head_{i,0} \rangle$  where  $head_{i,0}$  is the sentinel node of the wait-free queue.  $head_{i,j}$  represents a node with sequence number  $j$  in the wait-free queue.  $O_{i,j}.op_{j+1}()$  represents the execution of operation  $op_{j+1}()$  on  $O_{i,j}$ , and the resulting object  $O_{i,j+1}$  will contain the effects of  $op_{j+1}()$ . We define `curComb` as the Combined instance on which read-only operations execute.

► **Proposition 1.** *`curComb` can only transition between different Combined instances both protected by a shared lock.*

*Proof.* At the start of the execution, *curComb* references a Combined instance protected by a shared lock. The state transition of *curComb* occurs at line 61 between two Combined instances, referred as *lComb* and *newComb*. The lock associated with the *newComb* instance is acquired in exclusive mode at line 21 and is later downgraded to shared mode at line 50. It is not possible for *newComb* to be the *lComb* because *lComb* is protected by a shared lock and *newComb* had to be first acquired in exclusive mode. As such, *curComb* can only transition to a different Combined instance and that instance's lock is held in shared mode.  $\square$

► **Proposition 2.** *At most  $2t$  Combined instances are necessary to guarantee that an update operation will acquire an exclusive lock on one of the Combined instances.*

*Proof.* A process executing *applyUpdate()* will require at most two Combined instances at any given time, the acquisition of an exclusive lock at line 21 and a shared lock at line 31 or 54. Assuming the reader-writer trylock methods guarantee that no available Combined instance can remain inaccessible to all competing processes, this implies that any process that failed to acquire a lock in a Combined instance is sure that the instance is in use by a competing process. By induction, let's consider that processes  $q_1, \dots, q_{t-1}$  use  $2(t-1)$  Combined instances. The last process  $q_t$  will have available the last two Combined instances. Considering that process  $q_1$  releases the shared lock of *Comb<sub>i</sub>* and leaves the other in handover state. In a subsequent call to *applyUpdate()*, process  $q_1$  will acquire the exclusive lock on *Comb<sub>i</sub>* because this is the first available Combined instance when traversing the *combs[]* array, and the shared lock will be acquired on one of the  $2(t-1)$  Combined instances that can potentially be *curComb*. In the event that process  $q_t$  transitions *curComb* to one of its two available instances, then process  $q_1$  can acquire that Combined instance in shared mode but it will leave a precedent Combined instance available to be acquired in exclusive mode by process  $q_t$ . This shows that there will always be two Combined instances available to process  $q_n$ , the maximum it may need.  $\square$

► **Proposition 3.** *For any *Comb<sub>i</sub>*, an update operation with sequence  $l$  will transition atomically from  $\langle O_{i,j}, head_{i,j} \rangle$  to  $\langle O_{i,l}, head_{i,l} \rangle$ .*

*Proof.* Every Combined instance *Comb<sub>i</sub>* is protected by a reader-writer trylock, granting exclusive access at line 21 by Proposition 2 to only one process. This process will be allowed to mutate its state from the pair  $\langle O_{i,j}, head_{i,j} \rangle$  to a subsequent state. Only a process that is executing an update operation can acquire an exclusive lock on *Comb<sub>i</sub>*. Its update operation was previously appended to the queue and we assume the sequence number of the operation is  $l$ . Subsequently, the process will execute the statements from line 29 to 48, where the initial simulated object is  $O_{i,j}$ .  $O_{i,j}$

will be subjected to the execution of the sequence of operations  $op_k()$  where  $k = j+1, \dots, l$ , transitioning the simulated object to  $O_{i,l}$ . The traversal of the queue is required to be linearizable. The sequence of operations observed by a process traversing the queue is the same for all other processes. All concurrent mutative operations applied to object  $O$  were previously appended to the queue, and the queue establishes the partial history of the concurrent execution. The simulated object has now mutated to  $O_{i,l}$  and, at line 49, *Comb<sub>i</sub>* state will transition to  $\langle O_{i,j}, head_{i,j} \rangle$  where  $head_{i,l}$  represents the node containing the last operation applied to the simulated object  $O_{i,l}$ . Only after the transition to state  $\langle O_{i,l}, head_{i,l} \rangle$  is completed, will the Combined instance *Comb<sub>i</sub>* be made available to other processes, at line 50.  $\square$

► **Proposition 4.** **curComb* always transitions from *Comb<sub>i</sub>* to *Comb<sub>k</sub>*, with respective states  $\langle O_{i,j}, head_{i,j} \rangle$  and  $\langle O_{k,l}, head_{k,l} \rangle$ , where  $i \neq k$  and  $l > j$ .*

*Proof.* Proposition 4 follows from Proposition 1 and Proposition 3. In addition, the state transition can only occur (line 61) if  $head_{i,j}$  does not satisfy the condition at line 56. This condition guarantees that  $l > j$ .  $\square$

► **Lemma 1.** *An update operation with sequence number  $l$  can only return, after ensuring *curComb* transitions to a *Comb<sub>i</sub>* with state  $\langle O_{i,j}, head_{i,j} \rangle$  where  $l \leq j$ .*

*Proof.* An update operation with sequence number  $l$  will complete as soon as *applyUpdate()* returns (lines 26, 35, 69 or 73). After the exclusive lock is acquired for the Combined instance *Comb<sub>k</sub>* with state  $\langle O_{k,m}, head_{k,m} \rangle$  at line 24, it will validate if the sequence number of  $head_{k,m}$  is greater than or equal to  $l$ , implying that  $l \leq m$ . If *Comb<sub>k</sub>* was acquired in exclusive mode, then any previous update operation with sequence number  $m$  that updated *Comb<sub>k</sub>* with update operations until  $head_{k,m}$  had to guarantee that *curComb* was referencing a Combined instance *Comb<sub>i</sub>* with state  $\langle O_{i,j}, head_{i,j} \rangle$  where  $m \leq j$ . This proves that an update operation with sequence  $l$  can return at line 26 with the guarantee that *curComb* was at least referencing a Combined instance *Comb<sub>i</sub>* with state  $\langle O_{i,j}, head_{i,j} \rangle$  where  $l \leq j$ .

Execution from lines 30 to 41 occurs when the Combined instance acquired in exclusive mode requires a copy of *curComb*. The method *getCombined()* can only return null in two cases: in case *curComb* sequence  $j$  is higher than or equal to  $l$ ; otherwise, if after  $t$  trials it fails to acquire the shared lock of the current *curComb*, represented as a specific *Comb<sub>i</sub>* with state  $\langle O_{i,j}, head_{i,j} \rangle$ . This can only occur if *curComb* changed at least  $t$  times. By Proposition 4, *curComb* must be referencing a Combined instance with state  $\langle O_{k,m}, head_{k,m} \rangle$  where  $j+t \leq m$ . Assuming no operation can return before ensuring its operation is visible at *curComb* then  $l \leq j+t$ , implying that  $l \leq m$ . This proves that *curComb* must contain the update operation with sequence  $l$  after  $t$  transitions.

At line 69 the update operation returns after ensuring that *curComb* references a Combined instance with sequence number lower than *l* (line 56) and successfully transitions *curComb*, which by Proposition 4 will map to a Combined instance with state  $\langle O_{k,l}, head_{k,l} \rangle$  where  $i \neq k$  and  $l > j$ .

When the update operation returns after *t* failed attempts to acquire the shared lock of the current *curComb* at line 73, *curComb* must contain the update operation with sequence *l*.  $\square$

From Lemma 1, we can directly infer the following corollary.

► **Corollary 1.** *curComb always references a Combined instance with state  $\langle O_{i,j}, head_{i,j} \rangle$  where  $l - j \leq t$ , with *l* the sequence number at the tail of the wait-free queue.*

We now introduce the remaining lemmas that will allow us to prove linearizability of the CX universal construction.

► **Lemma 2.** *Given  $op_1$  and  $op_2$  two update operations on object *O*, if  $op_1 <_{hb} op_2$  and  $op_2$  belongs to *S* then  $op_1$  belongs to *S*, where *S* is a subhistory of *H* on *O*.*

*Proof.* Follows from Proposition 3.  $\square$

► **Lemma 3.** *Given  $op_u$  an update operation and  $op_r$  a read-only operation, both on object *O*, if  $op_u <_{hb} op_r$  then  $op_r$  will have to see the effects of  $op_u$  on *O*.*

*Proof.* If  $op_r$  execution accesses a Combined instance that does not contain the effects of  $op_u$ , then *curComb* has not yet transitioned to an instance that contains  $op_u$ . By Lemma 1, the  $op_u$  is only considered to take effect after *curComb* transitions to a Combined instance which *O<sub>i</sub>* contains the effects of  $op_u$ . This means that  $op_r$  could take place before  $op_u$ , implying  $op_r <_{hb} op_u$ , thus contradicting the initial assumption.  $\square$

► **Lemma 4.** *Given  $op_r$  a read-only operation and  $op_u$  an update operation, both on object *O*, if  $op_r <_{hb} op_u$  then  $op_r$  will not see the effects of  $op_u$  on *O*.*

*Proof.*  $op_u$  can only return after guaranteeing that *curComb* has transitioned to a Combined instance that contains the effects of  $op_u$ , by Lemma 1. Any read operation accesses only the Combined instance referenced by *curComb*. If  $op_r$  accesses a Combined instance that contains the effects of  $op_u$  then it would be possible to consider as if  $op_u$  had occurred before  $op_r$ , because the current *curComb* already contains  $op_u$ . This contradicts the definition of happens-before, if the response of  $op_u$  can occur before the invocation of  $op_r$  then  $op_u <_{hb} op_r$  which contradicts the initial assumption. Implying *curComb* can not contain the effects of  $op_u$  and, therefore,  $op_r$  will not see the effects of  $op_u$  over object *O*.  $\square$

► **Lemma 5.** *Given  $op_1$  and  $op_2$  two identical read-only operations on object *O*, if  $op_1 <_{hb} op_2$  then  $op_2$  returns the same result as  $op_1$ , unless an update operation  $op_u$  interleaves.*

*Proof.* By Lemma 1, only update operations can transition the state of *curComb*. All read operations access only the Combined instance referenced by *curComb*. If there is no update operation  $op_3$  interleaving between  $op_1$  and  $op_2$ , the read operations will necessarily access the same Combined instance yielding the same result.  $\square$

#### 4.1 Wait-Freedom

The *applyUpdate()* method has only one loop where the number of iterations is not predetermined (line 29). For it to terminate, the traversal of the wait-free queue must encounter the node containing the process' update operation. The process starts by appending its update operation with sequence number *l* to the wait-free queue and will proceed to acquire an exclusive lock for Combined instance *Comb<sub>i</sub>*. It is guaranteed by Proposition 2 that each process will always have available two Combined instances to execute, even if all other threads fail while holding two Combined instances locked, one in exclusive and another in shared mode. For the process to execute the loop at line 29, *Comb<sub>i</sub>*'s state must be  $\langle O_{i,j}, head_{i,j} \rangle$  where  $l > j$ , otherwise *applyUpdate()* would return at line 26. From Proposition 3, *Comb<sub>i</sub>*'s state will transition to  $\langle O_{i,l}, head_{i,l} \rangle$  in  $l - j$  iterations, unless a copy of object *O* is required (line 30). In case the process is unable to do a copy, it will return at line 35, thus terminating the loop. Otherwise, the copy from a Combined instance *Comb<sub>k</sub>* referenced by *curComb* with state  $\langle O_{k,m}, head_{k,m} \rangle$  is executed at line 40. The copy from *O<sub>k</sub>* is guaranteed to execute in a finite number of steps because *Comb<sub>k</sub>* is protected by a shared lock, which guarantees that no update operation is taking place during the copy procedure. After the copy is completed, *Comb<sub>i</sub>* will be in the state  $\langle O_{i,m}, head_{i,m} \rangle$ . The copy was performed from a Combined instance referenced by *curComb*, and Corollary 1 guarantees that  $l - m \leq t$ . Consequently, the loop at line 29 will iterate at most *t* times after the copy procedure. In all possible scenarios, the loop will always iterate a finite number of steps.

The *applyUpdate()* method also calls *enqueue()* at line 18 and the try-lock methods, *exclusiveTryLock()*, *exclusiveUnlock()*, *sharedTryLock()*, *sharedUnlock()*, *downgradeToHandover()* and *handoverUnlock()*. By definition, all these methods return in a finite number of steps, from which we conclude that *applyUpdate()* has wait-free progress. In addition, assuming the sequential copy of an object is bounded, then the *applyUpdate()* method is also wait-free bounded. The loop at line 29 is the only one that can be unbounded. The nodes of the wait-free queue are first retired to a thread-local circular buffer. Reclamation of the nodes is done once the thread-local circular buffer is full. Considering that *size<sub>circularbuffer</sub>* represents the size of the circular buffer where retired nodes are placed waiting to be latter reclaimed, then the maximum amount of iterations in the

loop at line 29 is bounded by:

$$size_{circbuff} \times t + t \quad (1)$$

The `applyRead()` method iterates for a maximum of `MAX_TRIES + t`. It calls `sharedTryLock()` and `sharedUnlock()`, which by definition return in a finite number of steps, resulting in wait-free progress.

## 5 Evaluation

We now present a detailed evaluation of CX and compare it with other state-of-the-art UCs and non-blocking data structures, using synthetic benchmarks. Our microbenchmarks were executed on a dual-socket 2.10 GHz Intel Xeon E5-2683 (“Broadwell”) with a total of 32 hyper-threaded cores (64 HW threads), running Ubuntu LTS and using gcc 8.3.

Besides CX, we used another wait-free UC, PSim [9]. We adapted the original implementation available on github. We have further designed and implemented an extension to PSim, that we call PSimOpt, where read-only operations have a different code path that allows them to scale, using a technique similar to the one we developed for CX. We also added a modified version of CX, called CXTimed, which restricts the amount of available Combined instances to four for a bounded period of time. For CXTimed, an update operation will spin on the first 4 Combined instances trying to acquire an exclusive lock while verifying if the update operation was executed by another thread. This restriction is imposed for a duration that is proportional to the time it takes to do a copy of the object. After that amount of time has elapsed and the update operation remains to be executed, the thread will acquire an exclusive lock on one of the  $2t$  instances. This can be seen as a blocking fast path with only 4 available Combined instances, that can always fallback to a slower path with wait-free progress [16]. This approach further reduces the amount of used replicas and object copies, because with high probability the first 4 instances are kept up-to-date.

Depending on the benchmark, we also compared with commonly available lock-free data structures. MagedHP is a Harris linked list set modified by Michael [18]. MagedHash is the hash table by Michael [18]. Natarajan is the relaxed tree by Natarajan and Mittal [20]. All implementations use Hazard Pointers [19].

All these data structures are *sets* and the microbenchmarks described next follow the same procedure. A set is filled with 1,000 keys and we randomly select doing either a lookup or an update, with a probability that depends on the percentage of updates for each particular workload. For a *lookup*, we randomly select two keys and call `contains()` for each; for an *update*, we randomly select one key and call `remove(key)`, and if the removal is successful, we re-insert the same key with a call to `add(key)`, thus maintaining the total number of keys in the set (minus any ongoing removals). Depending on the scenario, the procedure may be repeated for sets of

different key ranges. Each run takes 20 seconds, where a data point corresponds to the average of 5 runs. All implementations are available publicly.

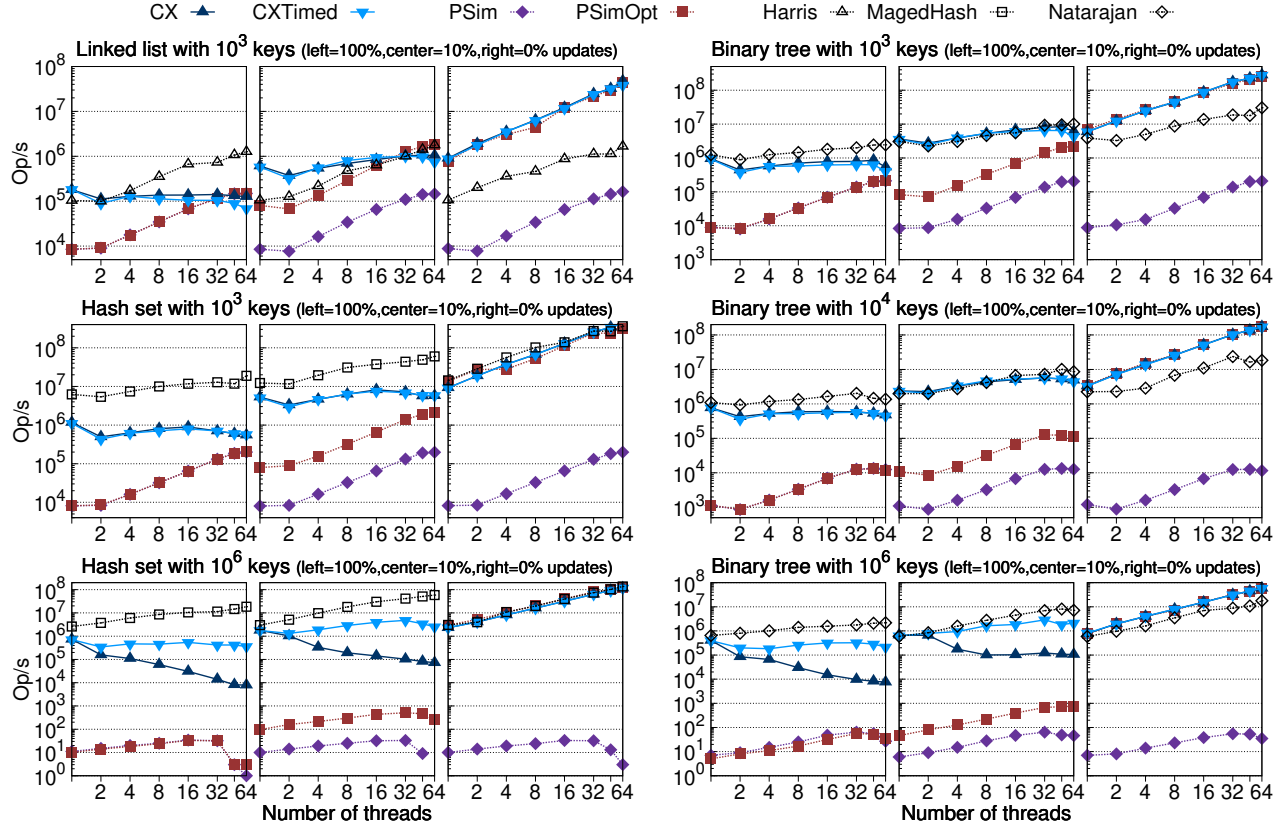
The results of our experiments are shown in Figure 3, with a log-log scale. As expected, a sorted linked list protected by CX is surpassed in write-mostly workloads by Maged-Harris’ lock-free set because of the serialization of all operations in the wait-free queue necessary to reach consensus. It is interesting to notice, however, that Maged-Harris algorithm is not able to outperform CX in scenarios with 10% updates or less. CX read operations do not require any pointer tracking during traversal because the data structure where the operation is executed is protected by a shared lock, which is not the case for traversals with Maged-Harris.

Let us now consider experiments with hash sets. The MagedHash algorithm uses a pre-allocated array of 1,000,000 buckets and its advantage over CX is significant, due to CX serializing all mutative operations, while updates on the MagedHash are mostly disjoint. However, CX is able to match MagedHash for the read-only scenario, because read operations have effectively parallel execution.

Regarding balanced trees, three different workloads were executed with  $10^3$ ,  $10^4$  and  $10^6$  keys, shown in Figure 3. Natarajan’s tree is a *non-balanced* tree and for that matter it requires a random fill of the keys. There are a few lock-free balanced trees in the literature [4], however, there is no known implementation with lock-free memory reclamation. Balanced trees, like the `std::set` sequential implementation we use in CX, do not impose any usage pattern. For write-intensive workloads, Natarajan tree can outperform CXTimed by a factor of 10x, in a one million key set. As the ratio of read operations increase, CXTimed performance gap reduces and on read mostly workloads is the best. CX and CXTimed have similar performance in most of the experiments, except when considering large data sets. The only difference between the two implementations is the backoff policy present in CXTimed. This backoff policy fosters the use of less replicas and reduces the number of required copies. As an example, Figure 4 shows how many times each replica was used (left plot), as well as the number of performed copies (right plot), for a one hour run of the benchmark described previously, in a tree with 1 million keys, with 32 threads concurrently executing update operations. The left histogram shows that CXTimed used only the first 7 replicas, with most of the execution done in the first 4 replicas. On the other hand, CX used all 64 replicas with a much more even usage pattern. During this execution, CXTimed performed 257 copies of the data set, while CX made almost two hundred thousand copies.

### 5.1 Concurrent Reading while Writing

The previous shown benchmarks have *mixed task* workloads, where a thread alternates between executing update operations and executing read-only operations. Such workloads



**Figure 3.** Left column (top to bottom): sets implemented using a linked list with  $10^3$  keys and using a hash table with  $10^3$  and  $10^6$  keys. Right column: sets implemented using the balanced binary search trees `std::set`, with  $10^3$ ,  $10^4$  and  $10^6$  keys. The results are presented with a logarithmic scale on both axes.

are fine for measuring the throughput of blocking data structures and blocking synchronization mechanisms, however, they are unsuitable to demonstrate starvation effects.

For example, if the synchronization mechanism is not starvation-free for read-only operations, the mixed task scenario typically ensures that all threads attempting a read-only operation will eventually succeed. A read-only operation will wait (block) for all the update operations to complete, and eventually all threads will be attempting read-only operations, and therefore succeed. This effect does not occur when each thread is continuously executing a read or an update operation.

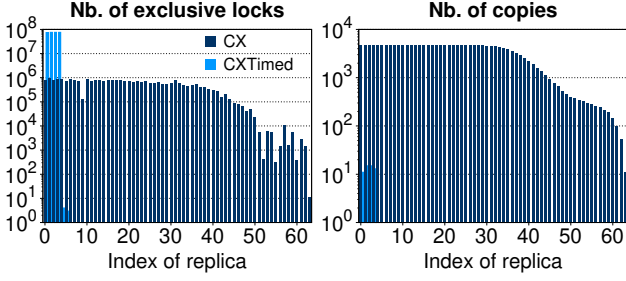
To evaluate the impact of continuous read and write operations in the overall performance, we designed a slightly modified benchmark with a tree of  $10^6$  keys, where half the threads are *dedicated updaters*, and the other half are *dedicated readers*. The update operation is similar to the one described in the previous section, while the read operation is a linearizable range query over  $10^3$  keys in the set, and therefore, takes longer than the update operation. Neither the Maged-Harris linked list set nor the Natarajan tree provide

linearizable read-only range queries. As can be seen in Figure 5, CXTimed does at least 3000× more update operations due to reducing the number of data structure copies when compared with PSimOpt, while CX does at least 200× compared to PSimOpt. For any of the data points in the update operations plot, PSim and PSimOpt do at most 35 operations per second.

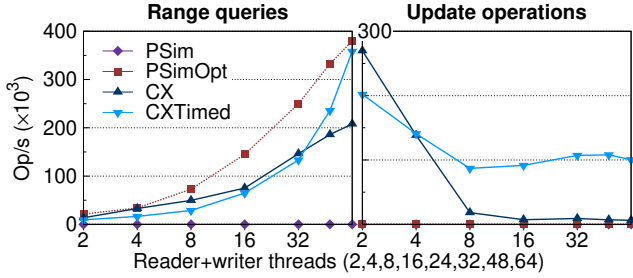
To conclude, the two CX implementations evaluated in this section give high scalability for read-mostly workloads regardless of the underlying data structure or workload user pattern. Its high throughput surpasses equivalent lock-free data structures, while providing wait-free progress and linearizable consistency for any operation. As for other UCs, PSim drags far below in all tested scenarios due to serializing read and write operations, even though it has been until now the best of the non-interposing UCs.

## 5.2 Memory Usage

For large data structures, the amount of memory required to execute the program can be a determining factor when choosing a more suitable concurrency synchronization. We conducted an experiment meant to evaluate the trade-off



**Figure 4.** Left histogram: number of times the lock of each replica was acquired in exclusive mode. Right histogram: number of copies per replica.

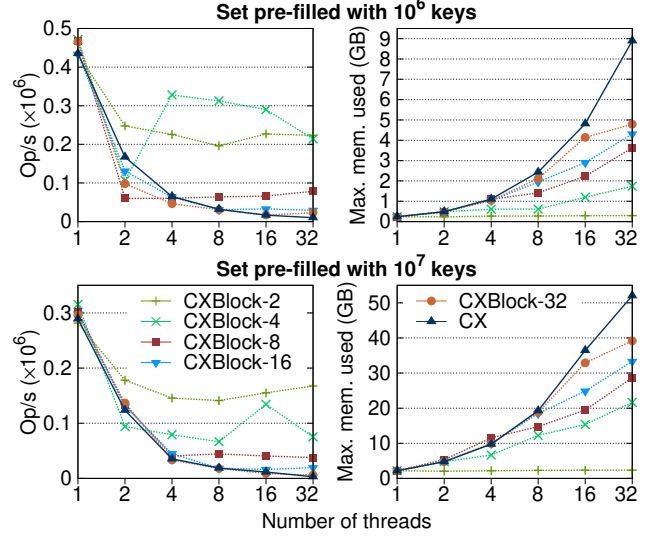


**Figure 5.** Half the threads are dedicated updaters and the other half dedicated readers executing range queries (key range:  $10^6$ , range query size:  $10^3$ ).

between memory usage and throughput. The experiment follows the same procedure as for update-only workloads, using the balanced binary search tree available in STL, with pre-filled trees of 1 and 10 million keys. The maximum memory usage is measured executing the same microbenchmark. Each data point of Figure 6 is the highest value of two runs, each run executing for 100 seconds.

We observe in Figure 6 that, unsurprisingly, the configuration with the lowest memory usage is when CX has *maxObjs* set to 2. It is only using two Combined instances with a constant maximum memory usage around 280 MB. When increasing the size of the data structures from  $10^6$  to  $10^7$  keys, we observe that memory usage grows as expected by a factor of ten, around 2 GB.

On the other hand, CX with wait-free progress (i.e., when *maxObjs* is set to 64) is the configuration with the highest memory requirements. As the number of threads grows, we observe both an increase in memory usage and a decrease in throughput, but this is compensated by the additional guarantees of resilience in case of thread failures. Our experiments also show that, for the tree with 1 million keys, CXBlock-4 sometimes achieves better performance than CXBlock-2. We can reach the conclusion that, if the application can relax the progress guarantees for update operations, then a suitable configuration would be to use up to 4 object instances.



**Figure 6.** Throughput vs. maximum memory usage with 100% updates for a pre-filled std::set of  $10^6$  keys (top) or  $10^7$  keys (bottom). CXBlock- $k$  is CX with *maxObjs* =  $k$ , CX is the wait-free UC with maximum number of objects 64.

This would provide a good trade-off between memory usage, throughput and progress.

## 6 Conclusion

The appeal of generic techniques like wait-free universal constructions (UC) stems from the difficulty in designing *hand-written* non-blocking data structures.

CX is the first non-instrumenting UC capable of transforming *any* sequential implementation of a data structure, with unforeseen method implementations, to be considered for multi-threaded applications with performance that surpasses hand-made lock-free implementations for read-intensive workloads.

Moreover, CX has integrated wait-free memory reclamation, a feature that most hand-written lock-free data structures do not provide. Using CX we have implemented the first wait-free binary *balanced* tree, showing that CX makes it possible to create new data structures for which no hand-made counterparts exist yet.

CX's huge leap in performance compared with previous UCs is due to the significant reduction of copy operations, where available replicas are instead reused and updated.

## Acknowledgments

We would like to thank our shepherd Erez Petrank for his help and guidance to better expose the contributions of our work. This work is supported in part by the Swiss National Science Foundation (SNSF) under project number 200021-178822/1.

## References

- [1] Yehuda Afek, Dalia Dauber, and Dan Touitou. 1995. Wait-Free Made Fast. In *Proceedings of the Twenty-Seventh Annual ACM Symposium on Theory of Computing (STOC '95)*. Association for Computing Machinery, New York, NY, USA, 538–547. <https://doi.org/10.1145/225058.225271>
- [2] James H. Anderson and Mark Moir. 1995. Universal Constructions for Multi-Object Operations. In *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing (PODC '95)*. Association for Computing Machinery, New York, NY, USA, 184–193. <https://doi.org/10.1145/224964.224985>
- [3] J. H. Anderson and M. Moir. 1999. Universal constructions for large objects. *IEEE Transactions on Parallel and Distributed Systems* 10, 12 (Dec 1999), 1317–1332. <https://doi.org/10.1109/71.819952>
- [4] Trevor Brown, Faith Ellen, and Eric Ruppert. 2014. A General Technique for Non-Blocking Trees. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '14)*. Association for Computing Machinery, New York, NY, USA, 329–342. <https://doi.org/10.1145/2555243.2555267>
- [5] Irina Calciu, Dave Dice, Yossi Lev, Victor Luchangco, Virendra J. Marathe, and Nir Shavit. 2013. NUMA-Aware Reader-Writer Locks. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '13)*. Association for Computing Machinery, New York, NY, USA, 157–166. <https://doi.org/10.1145/2442516.2442532>
- [6] Phong Chuong, Faith Ellen, and Vijaya Ramachandran. 2010. A Universal Construction for Wait-Free Transaction Friendly Data Structures. In *Proceedings of the Twenty-Second Annual ACM Symposium on Parallelism in Algorithms and Architectures (SPAA '10)*. Association for Computing Machinery, New York, NY, USA, 335–344. <https://doi.org/10.1145/1810479.1810538>
- [7] Andreia Correia and Pedro Ramalhete. 2018. Strong Trylocks for Reader-Writer Locks. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '18)*. Association for Computing Machinery, New York, NY, USA, 387–388. <https://doi.org/10.1145/3178487.3178519>
- [8] Faith Ellen, Panagiota Fatourou, Eleftherios Kosmas, Alessia Milani, and Corentin Travers. 2012. Universal Constructions That Ensure Disjoint-Access Parallelism and Wait-Freedom. In *Proceedings of the 2012 ACM Symposium on Principles of Distributed Computing (PODC '12)*. Association for Computing Machinery, New York, NY, USA, 115–124. <https://doi.org/10.1145/2332432.2332457>
- [9] Panagiota Fatourou and Nikolaos D. Kallimanis. 2014. Highly-Efficient Wait-Free Synchronization. *Theory Comput. Syst.* 55, 3 (2014), 475–520. <https://doi.org/10.1007/s00224-013-9491-y>
- [10] Maurice Herlihy. 1991. Wait-Free Synchronization. *ACM Trans. Program. Lang. Syst.* 13, 1 (1991), 124–149. <https://doi.org/10.1145/114005.102808>
- [11] Maurice Herlihy. 1992. A Methodology for Implementing Highly Concurrent Data Objects (Abstract). *Operating Systems Review* 26, 2 (1992), 12. <https://doi.org/10.1145/142111.964613>
- [12] Maurice Herlihy and J. Eliot B. Moss. 1993. Transactional Memory: Architectural Support for Lock-Free Data Structures. In *Proceedings of the 20th Annual International Symposium on Computer Architecture (ISCA '93)*. Association for Computing Machinery, New York, NY, USA, 289–300. <https://doi.org/10.1145/165123.165164>
- [13] Maurice Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (1990), 463–492. <https://doi.org/10.1145/78969.78972>
- [14] Amos Israeli and Lihu Rappoport. 1994. Disjoint-Access-Parallel Implementations of Strong Shared Memory Primitives. In *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing (PODC '94)*. Association for Computing Machinery, New York, NY, USA, 151–160. <https://doi.org/10.1145/197917.198079>
- [15] Alex Kogan and Erez Petrank. 2011. Wait-Free Queues with Multiple Enqueuers and Dequeuers. In *Proceedings of the 16th ACM Symposium on Principles and Practice of Parallel Programming (PPoPP '11)*. Association for Computing Machinery, New York, NY, USA, 223–234. <https://doi.org/10.1145/1941553.1941585>
- [16] Alex Kogan and Erez Petrank. 2012. A Methodology for Creating Fast Wait-Free Data Structures. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '12)*. Association for Computing Machinery, New York, NY, USA, 141–150. <https://doi.org/10.1145/2145816.2145835>
- [17] Leslie Lamport. 1977. Concurrent Reading and Writing. *Commun. ACM* 20, 11 (1977), 806–811. <https://doi.org/10.1145/359863.359878>
- [18] Maged M. Michael. 2002. High Performance Dynamic Lock-Free Hash Tables and List-Based Sets. In *Proceedings of the Fourteenth Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA '02)*. Association for Computing Machinery, New York, NY, USA, 73–82. <https://doi.org/10.1145/564870.564881>
- [19] Maged M. Michael. 2004. Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects. *IEEE Trans. Parallel Distrib. Syst.* 15, 6 (2004), 491–504. <https://doi.org/10.1109/TPDS.2004.8>
- [20] Aravind Natarajan and Neeraj Mittal. 2014. Fast Concurrent Lock-Free Binary Search Trees. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '14)*. Association for Computing Machinery, New York, NY, USA, 317–328. <https://doi.org/10.1145/2555243.2555256>
- [21] Gary L. Peterson. 1983. Concurrent Reading While Writing. *ACM Trans. Program. Lang. Syst.* 5, 1 (1983), 46–55. <https://doi.org/10.1145/357195.357198>
- [22] Pedro Ramalhete and Andreia Correia. 2017. POSTER: A Wait-Free Queue with Wait-Free Memory Reclamation. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '17)*. Association for Computing Machinery, New York, NY, USA, 453–454. <https://doi.org/10.1145/3018743.3019022>
- [23] Michel Raynal. 2017. Distributed Universal Constructions: a Guided Tour. *Bulletin of the EATCS* 121 (2017). <http://eatcs.org/beatcs/index.php/beatcs/article/view/482>
- [24] Nir Shavit and Dan Touitou. 1995. Software Transactional Memory. In *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing (PODC '95)*. Association for Computing Machinery, New York, NY, USA, 204–213. <https://doi.org/10.1145/224964.224987>
- [25] John D. Valois. 1995. Lock-Free Linked Lists Using Compare-and-Swap. In *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing (PODC '95)*. Association for Computing Machinery, New York, NY, USA, 214–222. <https://doi.org/10.1145/224964.224988>

## A Artifact Appendix

### A.1 Abstract

The artifact contains the source code for our universal constructions and the multiple benchmarks used in our evaluation. Compilation requires a C++ compiler with C++17 support. Building and running the benchmarks will result in plots similar to the ones in Figure 3, Figure 5 and the leftmost plots of Figure 6.

### A.2 Artifact check-list (meta-information)

- **Algorithm:** CX, CX-Timed, P-Sim and P-SimOpt
- **Program:** microbenchmarks
- **Compilation:** g++ 8.3
- **Binary:** g++-8
- **Run-time environment:** Ubuntu 18.10
- **Hardware:** any CPU with a C++ compiler supporting C++17
- **Execution:** sole user execution for 12 hours
- **Output:** text data files with the results of the throughput for each execution
- **Experiments:** makefile and manual steps
- **How much disk space required (approximately)?:** 22 MB
- **How much time is needed to prepare workflow (approximately)?:** 1 minute
- **How much time is needed to complete experiments (approximately)?:** 12 hours
- **Publicly available?:** yes
- **Code licenses?:** MIT License

### A.3 Description

#### A.3.1 How delivered

Available as open source under the MIT software license:  
<https://github.com/pramalhe/CX>

#### A.3.2 Hardware dependencies

None.

#### A.3.3 Software dependencies

We have tested building and running on a clean install of Ubuntu 18.10. This required a C++ compiler with support for the C++17 language and the make tool. It should work with any Linux distribution.

#### A.3.4 Data sets

None.

### A.4 Installation

The source code can be built using the make command.

### A.5 Experiment workflow

The benchmarks can be executed by using the make run command. This in turn will execute the following binaries:

set-ll-10k, set-tree-1k, set-tree-10k, set-tree-1m, set-hash-1k, set-hash-1m, set-tree-1m-dedicated, set-treeblocking-1m and set-treeblocking-10m.

### A.6 Evaluation and expected result

Linear scalability should be observed for the 0% update workloads. For other workloads the results may vary a bit depending on the hardware characteristics.

### A.7 Experiment customization

Additional parameters can be explored, such as different number of threads, different running times, larger or smaller key sets, larger or smaller key ranges (for set-tree-1m-dedicated) or different key types. Changing this parameters requires modifying the .cpp files in the graphs/ folder and rebuilding with make.

### A.8 Notes

None.

### A.9 Methodology

Submission, reviewing and badging methodology:

- <http://cTuning.org/ae/submission-20190109.html>
- <http://cTuning.org/ae/reviewing-20190109.html>
- <https://www.acm.org/publications/policies/artifact-review-badging>